Explaining Deep Reinforcement Learning using Shapley Additive Explanations

Nga Kiu (Jennifer) Zhang

***Abstract*- Deep reinforcement learning (DRL) is increasingly deployed in real-world applications, yet its opaque internal decision-making processes make it difficult to understand how these AI agents arrive at specific actions. This literature review evaluates SHAP (Shapley Additive Explanations)** [1]**, an explainable AI (XAI) framework derived from cooperative game theory that provides guaranteed fairness through axiomatic properties. While applying SHAP to DRL presents inherent challenges, it remains the most principled method for quantifying feature contributions in complex DRL models. We analyse the theoretical foundations of SHAP, its practical application to policy and value functions, and the limitations that arise when explaining sequential decision-making systems.**

I. Introduction

Applications of deep reinforcement learning (DRL) are increasingly deployed in our daily lives such as autonomous driving, robotics, and recommender systems. Unlike supervised learning models trained on static datasets, DRL agents learn by interacting with an environment and optimising behaviour through improving every iteration. A human-like learning curve makes DRL particularly suitable for many safety-critical decision-making tasks, such as autonomous vehicle control trained in simulation [2]. However, despite strong empirical performance, DRL models are often treated as black boxes, with limited transparency even to their designers. This lack of interpretability raises concerns about reliability, safety, and accountability, motivating the need for explainable approaches that allow users, developers, and regulators to understand and trust DRL decisions.

The Interpretability-Accuracy trade-off

Explainability aims to help humans understand why a model makes certain decisions, either through post-hoc explanation techniques or by designing inherently interpretable models. These approaches fall under the broader field of Explainable AI (XAI). A well-known challenge in XAI is the trade-off between interpretability and predictive performance: simple models such as linear regressions or small decision trees are transparent but often underperform on complex real-world data, whereas expressive models like neural networks achieve higher accuracy at the cost of interpretability [3].

Among all XAI techniques, this literature review focuses on SHAP (Shapley Additive Explanations), a method based on Shapley values, introduced in cooperative game theory, which provides a mathematically grounded way to ensure completeness and faithfulness. Shapley value measures the contribution of an individual player, or feature in machine learning, by averaging its marginal contribution across all possible feature coalitions [1]. While SHAP is well

established in supervised learning, applying it to DRL is more challenging due to the sequential and often stochastic nature of agent-environment interactions[4]. Despite these challenges, SHAP remains one of the most principled approaches for analysing DRL decisions.

This literature review first outlines the theoretical basis of SHAP, then illustrates how SHAP is applied in DRL, involving value functions and policy networks. Then it discusses the key limitations and existing mitigation approaches. Finally, we include two case studies to illustrate the type of practical insights SHAP can provide on DRL, focusing specifically on applications in Deep Q-Network (DQN) and policy gradient methods.

II. Background

This section introduces the Shapley value in cooperative games, followed by its properties and the variations of SHAP models.

A. Definitions

We define a model f that maps a set of input variables, or features, to an output. In the context of cooperative games, these features are treated as players, denoted by the set $F = \{1, \dots, M\}$. A coalition S is any subset of features $S \subseteq F$. The goal is to determine each player's contribution to the model's "payoff" (prediction).

B. Shapley Values

Intuitively, the Shapley value φ_i measures a player's average marginal contribution across all possible coalitions. It evaluates how much value a player adds when joining different subsets, ensuring a fair attribution independent of coalition order.

Formally, Shapley value of the player i is defined by [1]:

$$\varphi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} \left[f\left(s_{S \cup \{i\}}\right) - f(s_S) \right].$$

The sum iterates over all subsets S of players that do not contain player i, where F is the set of all players. The weighting factor,

$$\frac{|S|!(|F|-|S|-1)!}{|F|!},$$

assigns a weight to each subset S based on how frequently that subset would appear immediately before player i joins, in all possible orders. $|S|$ and $|F|$, denote the number of players in S and F, respectively. This weighting ensures an unbiased average over all coalition possibilities.

The term,

$$f(S \cup \{i\}) - f(S),$$

is the marginal contribution of the player i to the subset S. In machine learning, this corresponds to evaluating the model when feature i is included versus excluded.

Thus, the Shapley value of a feature i is the weighted average of its marginal contributions across all possible coalitions.

C. From Shapley Values to SHAP

SHAP (SHapley Additive exPlanations) applies the Shapley values to machine learning models by treating features as players in a cooperative game. In SHAP, the model's prediction is the value of the grand coalition involving all features, and the goal is to assign each feature a fair share of the prediction. SHAP adopts the additive explanation model introduced in the reference paper [1].

An explanation model in SHAP has the form:

$$g(z') = \varphi_0 + \sum_{i=1}^{M} \varphi_i z_i',$$

where z' is a binary vector indicating whether a feature is present (1) or not (0), φ_i is the Shapley value of feature i, and φ_0 is the baseline output of the model.

D. Three Properties of SHAP [1]

SHAP is uniquely appealing because it satisfies three key properties: local accuracy, missingness, and consistency. These properties are vital because they provide mathematical guarantees of completeness and faithfulness.

Local accuracy

$$f(x) = g(z')$$

Where f represents the original complex model, and g represents the simplified explanation model using SHAP. The original input x and simplified input z' are related through a mapping $x = h_x(z')$. This means that the explanation model must match the output of the original model for the simplified input being explained.

Missingness

$$z_i' = 0 \Rightarrow \varphi_i = 0$$

If a feature z_i' is absent, its SHAP value must be zero.

Consistency

Let $f_x(z') = f(h_x(z'))$ and $z'\backslash i$ denote setting $z_i' = 0$. For any two models f and f', if

$$f_x'(z') - f_x'(z'\backslash i) \geq f_x(z') - f_x(z'\backslash i)$$

for all inputs $z' \in \{0,1\}^M$, then $\varphi_i(f', x) \geq \varphi_i(f, x)$.

If a feature contributes more to the model in an updated version of the model, its SHAP value must not decrease. SHAP is the only additive method proven to satisfy this property.

E. Variations of SHAP Models

There are different variations of SHAP models, for example KernelSHAP, DeepSHAP and TreeSHAP. The key differences in model compatibility, runtime, and accuracy between these variations are summarized in Table 1.

	Kernel-SHAP	DeepSHAP	TreeSHAP
Model Compatibility	Any model	Deep neural networks	Tree models
Runtime	Slow $O(2^M)$	Fast $O(K \cdot N)$	Very Fast $O(TLD^2)$
Accuracy / Guarantees	Strong theoretical guarantees	Approximate; Baseline-dependent	Exact Shapley values
Notes	Model-agnostic	Uses DeepLIFT rules	Structure exploiting

Table 1: Comparing Variations of SHAP Models [1] [7]

KernelSHAP is the model-agnostic version that estimates Shapley values by sampling feature coalitions and fitting a locally weighted surrogate model around the prediction. It follows the same idea of local approximation used in LIME, but with a weighting scheme that is consistent with Shapley theory [1] [5].

DeepSHAP is specifically designed for deep neural networks and computes approximate Shapley values through backpropagation using DeepLIFT rules. It runs faster than KernelSHAP but depends on a chosen reference input, making the explanations sensitive to baseline selection. [6]

TreeSHAP provides exact and fast Shapley values for tree-based models by exploiting their structure. Although less common in DRL, it is useful when the policy or value

function is approximated with decision-tree ensembles [7].

III. SHAP on DRL

A. Markov Decision Process (MDP)

Most DRL algorithms assume the environment is stationary, meaning the transition and reward functions do not change over time. This assumption is supported with Markov Decision Process (MDP) stating that the future state only depends on the current state and action [2] [8]. MDP provide the mathematical framework for modelling sequential decision making, which supports DRL models.

Fig.1 below illustrates the standard agent-environment interaction in an MDP. At each timestep, the agent receives the current state and reward from the environment, selects an action to feed back into the environment, and the environment responds with a new state and reward. This forms a closed interaction loop.

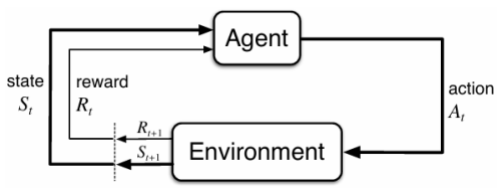

Fig.1: The agent-environment interaction in an MDP [2]

A finite MDP is defined by a tuple (S, A, T, R, γ), where S is a finite set of states (represented by state features). A is a finite set of actions available to the agent. $T(s'|s,a)$ is the transition probability, defining the probability of moving from state s to a new state s' after taking action a. $R(s,a)$ is a reward function, defining the immediate reward received after taking action a. $\gamma \in [0,1]$ is the discount factor that controls the importance of future rewards.

At each timestep t, the agent observes its current state s, selects an action a to move to a new state s' according to the transition probability $T(s'|\,a,s)$. The agent then receives a reward $r = R(s,a)$. Future rewards are discounted by a factor γ to compute long-term returns.

Our goal in reinforcement learning is to find the policy that maximises the expected long-term return. To formalise this, we define two key quantities: the state value function $V(s)$, saying how good it is to be in a state s, and the state-action value function $Q(s,a)$, saying how good it is to take action a in state s.

For an optimal agent, it must satisfy the Bellman Optimality Equations [2]:

$$Q^*(s,a) = \sum_{s'} T(s'|s,a)[r + \gamma \max_{a'} Q^*(s',a')],$$

$$V^*(s) = \max_a \sum_{s'} T(s'|s,a)[r + \gamma V^*(s')].$$

Value-based methods aim to learn $Q^*(s,a)$ or $V^*(s)$. The optimal policy is then derived by selecting the action with highest value of Q^* or V^*. Once learning converges, this policy becomes deterministic. Methods like Deep Q-Network (DQN) belong to this category.

Policy-based methods, different from value-based methods, represent the policy directly as a parameterised policy function $\pi_\theta(a|s)$, which represents a probability distribution of action a obtained from state s through a neural network mapping, and θ represent the weights in the neural network. The goal is to optimise the weights θ for next iteration to maximize the expected return $J(\theta)$, defined as:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_{t=0} \gamma^t r_{t+1}],$$

where r represents the immediate reward received at each timestep. Because the policy is represented as a probability distribution, the learned policy is generally stochastic. Methods like policy gradient fall under this category.

B. Explaining Deep Reinforcement Learning with SHAP

SHAP was originally designed for static prediction models such as classifiers or regressors. In these models, the input is a single feature vector, and SHAP measures how each feature contributes to the output [1]. In an MDP environment, at a single timestep, DRL network behaves like a standard predictive model.

Because of this, we can apply SHAP directly to DRL by treating states as features. SHAP estimates feature importance through a form of feature ablation; it systematically masks individual state features and observes the resulting variance in the model's output to quantify the specific impact of each input. If removing a feature produces a large change in the output, that feature is considered important for the agent's decision [9].

C. Restrictions of SHAP in Explaining DRL

Due to the sequential, dynamic, and often stochastic nature of DRL, several theoretical and practical limitations arise when applying SHAP.

Sequential Dynamics

Although SHAP can explain each decision at a single timestep in an MDP, it does not explain long-term behaviour or the full episode trajectory. In other words, SHAP cannot show how a state at timestep t influences the state or action at t+5, which aligns with the Markov property, where the distribution of future states depends only on the current state and action, not on earlier states [10].

Stochasticity

Some DRL algorithms (especially policy-based methods) use stochastic policies during training, meaning the policy network outputs a probability distribution over actions rather than a single deterministic choice. As a result, running the agent multiple times on the same state can lead to different sampled actions. Because of this randomness, SHAP cannot faithfully explain why a specific sampled action occurred [9]; however, SHAP can still explain the underlying deterministic policy output itself.

To mitigate the effect of stochasticity, different papers seem to agree on a similar approach: run many rollouts for each masked-feature coalition and average the results [11]. This lowers the variance caused by stochastic transitions, but it makes SHAP much more computationally expensive because it requires running multiple episodes [9].

For SHAP not being able to provide trajectory-level explanation, no widely adopted technique has been proposed to solve the sequential problem.

Faithfulness (False Positives and Negatives)

SHAP can produce false positives by attributing importance to correlated features that are not actually causal to the decision. Conversely, false negatives occur because SHAP provides only local, timestep-level explanations. It often fails to

credit features responsible for delayed, long-term rewards, effectively missing the agent's strategic foresight [11].

D. The Scope and Justification of SHAP in DRL

After stating the restrictions of SHAP in DRL, we start to ask, "What does SHAP actually explain?"

SHAP attributes importance to state features based on how they influence the output of the policy network but not the sampled action itself. The actions agent take are stochastic as they are sampled from a probability distribution, therefore the chosen action may differ from the highest probability action. However, the policy output (probabilities or Q-values) is deterministic for a given state. SHAP therefore explains why the policy prefers certain actions at that state, rather than why agent chose that specific action [4].

In practice, SHAP offers a local explanation of what information the agent used when making a decision at a single timestep. It shows which state features increased or decreased the preference for each action, revealing what the agent "attends to" in that moment. The paper Explaining Reinforcement Learning with Shapley Values [4] states that "Applying Shapley values to policies explains the contribution of state features to an agent's decisions, but not its performance."

The Cost vs. Consistency Trade-off

We must also address the fact that SHAP is computationally expensive due to stochasticity. While gradient-based methods like Saliency Maps or DeepLIFT are computationally cheaper and faster, they may violate axiomatic properties such as consistency, meaning they can assign lower importance to a feature even if its true effect increases. SHAP is preferred in high-stakes DRL (like autonomous driving) because its computational cost is the price paid for mathematical guarantees of faithfulness [12].

Even with these limitations, SHAP remains a theoretically well-grounded method for local explanations, as it satisfies the fairness properties of local accuracy, consistency and missingness, which help ensure that its attributions are mathematically sound. Although it cannot capture full trajectory-level behaviour in DRL, SHAP still provides useful insight into how the agent processes information at each individual decision point.

IV. Case Study 1: Traffic Signals

A. Traffic Signals Experiment [13]

This experiment is formulated as a real-time MDP that simulates traffic flow through a signalized roundabout. A policy gradient algorithm with a neural network policy is used because the state space is continuous and high-dimensional. The simulation runs for an hour with a timestep of 10 seconds. At each timestep, detectors are positioned at every entry and exit of a part of the roundabout as shown in Fig.2, recording the number of vehicles passing through, forming a matrix of traffic counts representing the current state. An action corresponds to selecting one predefined traffic-light phase from Table 2, where each phase specifies which entry arms receive Green, Amber or Red signals. The reward is defined as the number of vehicles exiting the roundabout during that timestep.

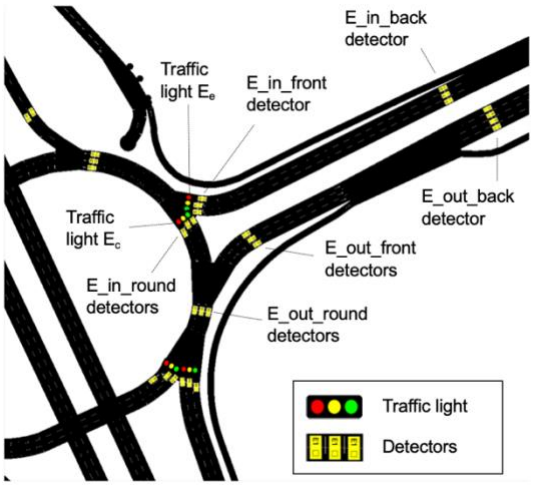

Fig.2: Illustration of the detectors and traffic lights for the East section on the roundabout. [13]

Action	Entry lanes	Circulatory lanes
a_1	$G(N_e); R(W_e, S_e, E_e)$	$G(W_c, S_c, E_c); R(N_c)$
a_2	$G(W_e); R(S_e, E_e, N_e)$	$G(S_c, E_c, N_c); R(W_c)$
a_3	$G(S_e); R(E_e, N_e, W_e)$	$G(E_c, N_c, W_c); R(S_c)$
a_4	$G(E_e); R(N_e, W_e, S_e)$	$G(N_c, W_c, S_c); R(N_c)$
a_5	$G(N_e, S_e); R(W_e, E_e)$	$G(W_c, E_c); R(N_c, S_c)$
a_6	$G(W_e, E_e); R(N_e, S_e)$	$G(N_c, S_c); R(W_c, E_c)$
a_7	$G(N_e), A(S_e); R(W_e, E_e)$	$G(W_c, E_c); A(S_c); R(N_c)$
a_8	$G(S_e), A(N_e); R(W_e, E_e)$	$G(W_c, E_c); A(N_c); R(S_c)$
a_9	$G(W_e), A(E_e); R(N_e, S_e)$	$G(N_c, S_c); A(E_c); R(W_c)$
a_{10}	$G(E_e), A(W_e); R(N_e, S_e)$	$G(N_c, S_c); A(W_c); R(E_c)$
a_{11}	$R(N_e, W_e, S_e, E_e)$	$G(N_c, W_c, S_c, E_c)$

Table 2: A table of Traffic-light Phases [13]

During training, the simulation is executed over multiple episodes so that the policy-gradient method can update the neural-network weights θ. The policy $\pi_\theta(a|s)$ is adjusted in the direction that increases the expected return.

After training, to give a global understanding, the authors analyse the learned policy by averaging the traffic detector counts and the action probabilities across the best one-hour episode. The policy shows a strong preference for phases that give a green signal to the East and West approaches, which is consistent with their higher measured traffic volumes. Here, green means the lane can fully proceed, while amber indicates a yield condition where vehicles may proceed only if no conflicting traffic is present.

However, the policy also displays an asymmetric behaviour that cannot be explained by traffic volumes alone. In some phases, the policy tends to prioritise action a_9: $G(W_e), A(E_e); R(N_e, S_e)$ (signal green to the West entry and signal amber to the East entry), than action a_{10}: $G(E_e), A(W_e)$; $R(N_e, S_e)$, even though the East and West arms have similar arrival rates. These asymmetric phase preferences suggest that the neural network policy is exploiting more subtle patterns that are not visible from aggregated traffic counts alone.

B. Applying SHAP [1][13]

To understand the agent's decisions at individual timesteps, the authors apply KernelSHAP to the policy output $\pi_\theta(a|s)$, to estimate how each detector contributes to increasing or decreasing the probability of selecting a traffic-light phase. Positive SHAP values indicate features that raise a phase's probability, while negative values suppress it.

The SHAP analysis shows that detector importance is highly uneven across the network. For example, the North detectors consistently have near-zero contribution, aligning with the averaged count matrix showing that the North entry has very low traffic.

SHAP also clarifies the asymmetric behaviour noted earlier. It shows that the South circulatory lane gives strong negative contribution to the East-by-West phase. Since the averaged count matrix indicates that the South circulatory lane is frequently congested, this phase becomes unlikely to be selected whenever congestion is present. In contrast, the corresponding West-by-East phase does not receive the same negative influence.

By computing the Shapley values of each detector, we measure how each state feature

contributes (positively or negatively) to the choice of each traffic-light phase, revealing policy preference that could not be observed simply from a global average alone.

V. Case Study 2: Autonomous Vehicle

A. Experiment setup [14]

This experiment trains a deep Q-learning model (DQN) for autonomous vehicle longitudinal control (i.e., whether the vehicle should accelerate). DeepSHAP is applied for both local explanation and global explanation, to explain the feature-level reasoning behind the agent's driving decisions.

B. Apply SHAP [14]

The author applied SHAP both locally (per timestep) and globally (over a trajectory).

Local Explanation

At each timestep, SHAP is computed on the Q-values. For each action a, SHAP explains the Q-function $Q(s, a)$, and each Shapley value indicates how much a feature contributes to increasing or decreasing the Q-value of that action. For example, at one particular timestep, SHAP shows that relative distance largely contributes to the acceleration action, while car speed and relative velocity contribute less. This result aligns with normal driving behaviour as a relatively large distance makes acceleration more desirable, because there is enough space to safely increase speed.

Global Explanation

Global SHAP explanations summarize feature importance across a trajectory. The global SHAP analysis suggested a positive correlation between vehicle speed and the tendency to accelerate, while this result normally contradicts general safe driving principles. This finding highlighted a crucial utility: SHAP successfully exposed an erroneous and potentially unsafe pattern learned by the model during training [14]. However, this result simultaneously illustrates the inherent challenge of applying SHAP globally in sequential decision-making systems. Since DRL decisions are dependent on long-term reward, averaging feature contributions over an entire trajectory could lead to seemingly non-intuitive or inconsistent explanations.

This experiment demonstrates that SHAP provides consistent and meaningful local explanations for individual decisions, but could be less consistent when encountering global explanation, highlighting the challenge of interpreting DRL policies over entire trajectories.

VI. Limitations and Future Directions

As previously discussed, there remain unresolved challenges in the state of the art (SOTA) when applying SHAP to deep reinforcement learning (DRL). SHAP provides explanations for a single decision, whereas reinforcement learning is inherently sequential. Several studies argue that DRL decisions cannot be explained in isolation because DRL decision making involves long-term reward, meaning that the value of a state also depends not only on its immediate reward but also on its future rewards. SHAP however only calculates the contribution of a state feature on a single timestep, which cannot represent the sequential-based part of DRL [15].

Beyond temporal concerns, SOTA research struggles with the issue of off-manifold which assumes feature independence. While this assumption is used in earlier SHAP-based studies, real-world DRL states lie on a structured manifold, which often possesses correlations in features. When SHAP samples feature coalitions that never occur in the environment, the resulting attributions fail to reflect true agent dynamics, leading to misleading explanations. While emerging methods like Shapley Values for Explaining Reinforcement Learning (SVERL) [4] attempt to address this, maintaining fidelity to the environment manifold remains an open challenge in the field.

VII. Conclusion

SHAP provides mathematically grounded and locally faithful explanations for individual DRL decisions, revealing which state features influence an agent's policy or value estimates. However, there are some restrictions: not being able to explain full trajectories, high computational power, and risk of creating unrealistic state due to off-manifold. Evidence from the sample cases shows that SHAP is reliable for single-step insight but having chance of being inconsistent when aggregated across time. Moving forward, XAI research needs to increase its focus on DRL, as most explainability work still targets standard deep learning models, even though DRL is now being used in real systems like warehouse robots and driverless cars. As DRL becomes more complex, progress in long-term temporal explanations, managing computational complexity and manifold-aware masking (like SVERL) will be essential so that people can safely understand and trust these technologies.

VIII. References

[1] Scott M. Lundberg, Su-In Lee, "A Unified Approach to Interpreting Model," 31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA, 2017.

[2] Richard S. Sutton, Andrew G. Barto, Reinforcement Learning: An Introduction (Second Edition), The MIT Press, 2015.

[3] G. A. Vouros, "Explainable Deep Reinforcement Learning: State of the Art and Challenges," ACM, 2022.

[4] Daniel Beechey, Thomas M. S. Smith, Ozgur Simsek, "Explaining Reinforcement Learning with Shapley Values," arXiv, Proceedings of the 40th International Conference on Machine Learning.

[5] Marco Tulio Ribeiro, Sameer Singh, Carlos Guestrin, ""Why Should I Trust You?" Explaining the Predictions of Any Classifier," KDD, San Francisco, CA, USA, 2016.

[6] Sindre Benjamin Remman, Anastasios M. Lekkas, "Robotic Lever Manipulation using Hindsight Experience Replay and Shapley Additive Explanations," arXiv, 2021.

[7] Scott M. Lundberg, Gabriel G. Erion, Su-In Lee, "Consistent Individualized Feature Attribution for Tree Ensembles," arXiv, 2019.

[8] A. D. Ranga Shaarad Ayyagari, "Markov Decision Processes Under External Temporal Processes," arXiv, 2024.

[9] Heuillet, Alexandre and Couthouis, Fabien and Díaz-Rodríguez, Natalia, "Collective eXplainable AI: Explaining Cooperative Strategies and Agent Contribution in Multiagent Reinforcement Learning with Shapley Values," *IEEE Computational Intelligence Magazine,* vol. 17, pp. 59-71, 2022.

[10] Venkat N. Gudivada, Dhana Rao, Vijay V. Raghavan, "Handbook of Statistics," in *Chapter 9 - Big Data Driven NLP Research and Applications*, 2015, pp. 207-208.

[11] Zelei Cheng, Jiahao Yu, Xinyu Xing, "A Survey on Explainable Deep Reinforcement Learning," arXiv, 2025.

[12] Anna Stubbin, Thompson Chyrikov, Jim Zhao, and Christina Chajo, "The Limits of Perception: Analyzing Inconsistencies in Saliency Maps in XAI," arXiv, 2024.

[13] Stefano Giovanni Rizzo, Giovanna Vantini and Sanjay Chawla, "Reinforcement Learning with Explainability for Traffic Signal Control," IEEE Intelligent Transportation Systems Conference (ITSC), Auckland, NZ, 2019.

[14] Meng Li, Zhihao Cui, Yulei Wang, Yanjun Huang, Hong Chen, “An Explainable Q-Learning Method for Longitudinal Control of Autonomous Vehicles,” IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, VOL. 26, NO. 3, 2025.

[15] Yuyao Wang, Masayoshi Mase, Masashi Egi, “Attribution-based Salience Method towards Interpretable Reinforcement Learning,” arXiv.